School 42 Brussels - Automation Testing Workshop

Complete Workshop Manual & Challenge Guide

Date: May 19, 2026
Duration: 4 Hours Workshop
Prepared by: Deloitte Quality Engineering Team

Pre-Workshop Setup

Pre-Requisite Software

Node.js: JavaScript runtime environment Download from Node.js — Download Node.js® NPM: Node package manager for downloading libraries

Install globally using:

```
npm install -g npm
```

To confirm Node.js and NPM are installed, run:

```
node –v & npm -v
```

Visual Studio Code: IDE optimized for Playwright automation Download from Download Visual Studio Code - Mac, Linux, Windows

Playwright Setup

Playwright: Test automation framework Install using:

```
npm install -g playwright
```

TypeScript: programming language for this session Install using:

```
npm install -g typescript
```

Verify both installations completed successfully:

```
playwright -V
tsc –v
```

Create a new project directory for Playwright, then go to it:

```
mkdir my-playwright-project
cd my-playwright-project
```

Use below command to scaffold a new project with sample tests, configuration, and browser support.

```
npm init playwright@latest
```

When prompted, provide these answers:

```
- Language: TypeScript
- Tests directory: tests (standard folder name)
- GitHub Actions: n
- Playwright browsers: Y
```

Running Playwright Tests from the terminal

Run Playwright tests efficiently from the terminal using simple commands to target specific files, browsers, and modes for faster debugging and execution.

• Run a specific test file:

```
npx playwright test tests/<test-file-name>
```

• Run all tests:

```
npx playwright test
```

• Run tests in headed (visual) mode:

```
npx playwright test --headed
```

• Run tests in a specific browser (e.g., Chromium):

```
npx playwright test --project=chromium
```

• Combine options for quick debugging:

```
npx playwright test tests/<test-file-name> --headed --project=chromium
```

The most up-to-date list of commands and arguments available on the CLI can always be retrieved via:

```
npx playwright –help
```

Find more information at Command line | Playwright

Today's Website

Will be using the https://bitheap.tech ‘fake’ webshop to learn Playwright. Create an account through Register – be sure to capitalise the first letter of your username to avoid any issues with capitalization later on. Log in, explore the different pages, and try purchasing products to familiarise yourself with the website’s functionalities and user interface.

Challenge 1: Login Functionality Testing

Objective

Write a Playwright test script that automates the login process for bitheap.tech.

Challenge 1 Checklist

- Created `tests/challenge1.spec.ts`
- Test navigates to bitheap.tech
- Test clicks login link
- Test fills username and password
- Test clicks login button
- Test verifies welcome message
- Test takes screenshot
- Test runs successfully with `npx playwright test`

Step-by-Step Guide

1. Create Test File

In VS Code, create a new file: `tests/challenge1.spec.ts`

2. Write the Test

```
import { test, expect } from '@playwright/test';

test('login', async ({ page }) => {
  // Navigate to the login page URL.
  await page.goto('https://bitheap.tech');
  await page.click('#menu-item-2330');

  // Enter a valid username and password.
  await page.locator('[name="xoo-el-username"]').fill('YOUR_USERNAME_HERE');
  await page.locator('[name="xoo-el-password"]').fill('YOUR_PASSWORD_HERE');

  // Click the login button.
  await page.locator('xpath=/html/body/div[8]/div[2]/div/div/div[2]/div/div/div[2]/div/form/button').click();

  // Verify successful login by checking for a specific element on the landing.
  await expect(page.locator('css=#menu-item-2333 > a')).toHaveText('Hello, YOUR_USERNAME_HERE');

  // Take screenshot for documentation
  await page.screenshot({ path: 'login-test.png' });
});
```

3. Run the Test

```
# Run the specific test
npx playwright test tests/challenge1.spec.ts

# Run in headed mode (see the browser)
npx playwright test tests/challenge1.spec.ts --headed

# Run in specific browser
npx playwright test tests/challenge1.spec.ts --project=chromium --headed
```

Challenge 2: Reusable Functions

Objective

Convert the login test into a reusable function that can be used across multiple tests.

Challenge 2 Checklist

- Updated `tests/challenge1.spec.ts`
- Implemented `authenticate()` function
- Function takes page, username, password as parameters
- Function verifies login with assertion
- Test uses the function
- Test runs successfully

Step-by-Step Guide

1. Refactor Challenge 1 to Add Function

Update `tests/challenge1.spec.ts` to include a reusable authenticate function:

```
import { test, expect } from '@playwright/test';

async function authenticate(page, username, password) {
  // Navigate to the login page URL.
  await page.goto('https://bitheap.tech');
  await page.click('#menu-item-2330');

  // Enter a valid username and password.
  await page.locator('[name="xoo-el-username"]').fill(username);
  await page.locator('[name="xoo-el-password"]').fill(password);

  // Click the login button.
  await page.locator('xpath=/html/body/div[8]/div[2]/div/div/div[2]/div/div/div[2]/div/form/button').click();

  // Verify successful login by checking for a specific element on the landing.
  await expect(page.locator('css=#menu-item-2333 > a')).toHaveText(`Hello, ${username}`);
}

test('login', async ({ page }) => {
  await authenticate(page, 'YOUR_USERNAME_HERE', 'YOUR_PASSWORD_HERE');
});
```

2. Run the Test

```
npx playwright test tests/challenge1.spec.ts --headed
```

Challenge 3: Add to Cart & Verify

Objective

Create a test that adds a product to the cart and verifies the cart contents.

Challenge 3 Checklist

- Created `tests/challenge3.spec.ts`
- Test authenticates user
- Test navigates to shop
- Test navigates to product
- Test adds 3 of the product to cart
- Test navigates to cart
- Test verifies 3 items of the product in cart
- Test runs successfully

Step-by-Step Guide

1. Create the Test File

Create `tests/challenge3.spec.ts`:

```
import { test, expect } from '@playwright/test';

async function authenticate(page, username, password) {
  await page.goto('https://bitheap.tech');
  await page.click('#menu-item-2330');

  await page.locator('[name="xoo-el-username"]').fill(username);
  await page.locator('[name="xoo-el-password"]').fill(password);

  await page.locator('xpath=/html/body/div[8]/div[2]/div/div/div[2]/div/div/div[2]/div/form/button').click();

  await expect(page.locator('css=#menu-item-2333 > a')).toHaveText(`Hello, ${username}`);
}

test('add to cart and verify cart contents', async ({ page }) => {
  // Authenticate user
  await authenticate(page, 'YOUR_USERNAME_HERE', 'YOUR_PASSWORD_HERE');

  // Navigate to Shop page
  await page.locator('#menu-item-1310').click();

  // Navigate through pages until the desired product link is visible
  while (!(await page.getByText('Useful ChatGPT Prompts').isVisible())) {
    await page.getByRole('link', { name: '→' }).click();
    await page.waitForLoadState('domcontentload');
  }

  // Select product, set quantity, and add to cart
  await page.getByAltText('Useful ChatGPT Prompts').click();
  await page.getByRole('spinbutton', { name: 'Product quantity' }).click();
  await page.getByRole('spinbutton', { name: 'Product quantity' }).fill('3');
  await page.getByRole('button', { name: '+ Add to cart' }).click();

  // Navigate to cart and verify contents
  await page.locator('xpath=/html/body/nav/div[1]/div[3]/div/a').click();
  await expect(page.locator('body')).toContainText('Useful ChatGPT Prompts');
  await expect(page.getByRole('cell', { name: 'Useful ChatGPT Prompts quantity' }).getByLabel('Product
quantity')).toHaveValue('3');
});
```

2. Run the Test

```
npx playwright test tests/challenge3spec.ts --headed
```

Challenge 4: Test Environment Cleanup

Objective

Use hooks to ensure the cart is empty before each test runs.

Challenge 4 Checklist

- Created `tests/challenge4.spec.ts`
- Implemented `test.beforeEach()` hook
- Hook authenticates user
- Hook navigates to cart
- Hook empties cart using for loop
- Test reuses code from Challenge 3
- Test runs successfully
- Cart is empty before test

Step-by-Step Guide

1. Create Test with Hooks

Create `tests/challenge4.spec.ts`:

```
import { test, expect } from '@playwright/test';

async function authenticate(page, username, password) {
  await page.goto('https://bitheap.tech');
  await page.click('#menu-item-2330');

  await page.locator('[name="xoo-el-username"]').fill(username);
  await page.locator('[name="xoo-el-password"]').fill(password);

  await page.locator('xpath=/html/body/div[8]/div[2]/div/div/div[2]/div/div/div[2]/div/form/button').click();

  await expect(page.locator('css=#menu-item-2333 > a')).toHaveText(`Hello, ${username}`);
}

test.describe('Shopping Cart Tests', () => {
  // This runs before each test
  test.beforeEach(async ({ page }) => {
    // Authenticate user
    await authenticate(page, 'YOUR_USERNAME_HERE', 'YOUR_PASSWORD_HERE');

    // Navigate to cart
    await page.locator('xpath=/html/body/nav/div[1]/div[3]/div/a').click();

    // Wait for cart page to load
    await page.waitForLoadState('domcontentloaded');

    // Empty the cart
    const amountInCart = await page.getByRole('spinbutton', { name: 'Product quantity' }).count();

    // Loop through each product and remove it
    for (let i = 0; i < amountInCart; i++) {

      // Click it
      await page.getByRole('spinbutton', { name: 'Product quantity' }).click();

      // Set quantity to 0
      await page.getByRole('spinbutton', { name: 'Product quantity' }).fill('0');

      // Press Enter to confirm
      await page.getByRole('spinbutton', { name: 'Product quantity' }).press('Enter');

      // Wait for cart to update
      await page.waitForLoadState('load');
    }
  });
```

```
  // Test: add to cart and verify cart contents
  test('add to cart and verify cart contents', async ({ page }) => {
    await page.locator('#menu-item-1310').click();

    while (!(await page.getByText('Useful ChatGPT Prompts').isVisible())) {
      await page.getByRole('link', { name: '→' }).click();
      await page.waitForLoadState('domcontentload');
    }

    await page.getByAltText('Useful ChatGPT Prompts').click();
    await page.getByRole('spinbutton', { name: 'Product quantity' }).click();
    await page.getByRole('spinbutton', { name: 'Product quantity' }).fill('3');
    await page.getByRole('button', { name: '+ Add to cart' }).click();

    await page.locator('xpath=/html/body/nav/div[1]/div[3]/div/a').click();
    await expect(page.locator('body')).toContainText('Useful ChatGPT Prompts');
    await expect(page.getByRole('cell', { name: 'Useful ChatGPT Prompts quantity' }).getByLabel('Product
quantity')).toHaveValue('3');
  });
});
```

2. Run the Test

```
npx playwright test tests/challenge4.spec.ts --headed
```

Challenge 5: Page Object Model (POM)

Objective

Refactor the test code using the Page Object Model pattern for better maintainability and readability.

Challenge 5 Checklist

- Created relevant pages
- Created `tests/challenge5.spec.ts`
- All page classes extend BasePage
- Test uses page objects
- Test is clean and readable
- Test runs successfully

Step-by-Step Guide

1. Create Base Page Class

Create `tests/pages/BasePage.ts`:

```
import { Locator, Page } from "@playwright/test"

export class BasePage {
  // Class properties
  private shopButton: Locator;
  private loginButton: Locator;
  private cartButton: Locator;
  private logOutButton: Locator

  // Constructor
  constructor(protected page: Page) {
    this.shopButton = page.locator("#menu-item-1310");
    this.loginButton = page.locator("#menu-item-2330");
    this.cartButton = page.locator("xpath=/html/body/nav/div[1]/div[3]/div/a");
    this.logOutButton = page.locator("#menu-item-2332").getByRole("link", { name: "Logout" });
  }

  // Open homepage
  async launch() {
    await this.page.goto("https://bitheap.tech", { waitUntil: "domcontentloaded" });
  }

  // Accept cookie consent
  async acceptCookies() {
    try {
      await this.page.getByRole("button", { name: "Accept All" }).waitFor({ timeout: 2000 });
      await this.page.getByRole("button", { name: "Accept All" }).click();
      await this.page.getByRole("button", { name: "Consent", exact: true }).click();
    } catch (error) {
      await this.page.getByRole("button", { name: "Consent", exact: true }).click();
    }
  }

  async navigateToShop() {
    await this.shopButton.click();
  }

  async navigateToLogin() {
    await this.loginButton.click();
  }

  async navigateToCart() {
    await this.cartButton.click();
  }

  async logout() {
    await this.logOutButton.click();
  }
}
```

2. Create Login Page Class

Create `tests/pages/LoginPage.ts`:

```
import { BasePage } from "./BasePage";
import { Locator, Page, expect } from "@playwright/test";

export class LoginPage extends BasePage {
  // Class properties
  private usernameField: Locator
  private passwordField: Locator
  private signInButton: Locator

  // Constructor
  constructor(page: Page) {
    super(page);
    this.usernameField = page.locator("[name='xoo-el-username']");
    this.passwordField = page.locator("[name='xoo-el-password']");
    this.signInButton = page.locator("xpath=/html/body/div[8]/div[2]/div/div/div[2]/div/div/div[2]/div/form/button");
  }

  // Perform login with given credentials
  async authenticate(username: string, password: string) {
    await this.navigateToLogin();

    await this.page.waitForLoadState("load");
    await this.usernameField.fill(username);
    await this.passwordField.fill(password);
    await this.signInButton.click();
    await this.page.waitForLoadState("domcontentloaded");
    await expect(this.page.locator("css=#menu-item-2333 > a")).toHaveText(`Hello, ${username}`);
  }
}
```

3. Create Shop Page Class

Create `tests/pages/ShopPage.ts`:

```
import { Locator, Page } from "@playwright/test";
import { BasePage } from "./BasePage";

export class ShopPage extends BasePage {
  // Class properties
  private nextPageArrow: Locator

  // Constructor
  constructor(page: Page) {
    super(page);
    this.nextPageArrow = page.getByRole("link", { name: "→" });
  }

  // Navigate through shop pages to find and select a product by name
  async goToProduct(productName: string) {
    await this.navigateToShop();
    while (!(await this.page.getByText(productName).isVisible())) {
      await this.nextPageArrow.click();
      await this.page.waitForLoadState("domcontentloaded");
    }
    await this.page.getByAltText(productName).click();
  }
}
```

4. Create Product Page Class

Create `tests/pages/ProductPage.ts`:

```
import { Locator, Page } from "@playwright/test";
import { BasePage } from "./BasePage";

export class ProductPage extends BasePage {
  // Class properties
  private quantityInput: Locator;
  private addToCartButton: Locator;

  // Constructor
  constructor(page: Page) {
    super(page);
    this.quantityInput = page.getByRole("spinbutton", { name: "Product quantity" });
    this.addToCartButton = page.getByRole("button", { name: "+ Add to cart" });
  }

  // Method to set the product quantity
  async setProductQuantity(quantity: number) {
    await this.quantityInput.click();
    await this.quantityInput.fill(quantity.toString());
  }

  // Method to add the product to the cart
  async addToCart() {
    await this.addToCartButton.click();
  }
}
```

5. Create Cart Page Class

Create `tests/pages/CartPage.ts`:

```
import { Locator, Page, expect } from "@playwright/test";
import { BasePage } from "./BasePage";

export class CartPage extends BasePage {
  // Class properties
  private quantityInput: Locator

  // Constructor
  constructor(page: Page) {
    super(page);
    this.quantityInput = page.getByRole("spinbutton", { name: "Product quantity" });
  }

  // Verify product is in cart with expected quantity
  async verifyProductInCart(productName: string, expectedQuantity: number) {
    await this.navigateToCart();
    await this.page.waitForLoadState("domcontentloaded");
    await expect(this.page.locator("body")).toContainText(productName);
    await expect(
      this.page.getByRole("cell", { name: `${productName} quantity` }).getByLabel("Product quantity")
    ).toHaveValue(expectedQuantity.toString());
  }

  // Empty the cart by setting all product quantities to zero
  async emptyCart() {
    await this.navigateToCart();
    await this.page.waitForLoadState("domcontentloaded");
    const amountInCart = await this.quantityInput.count();
    for (let i = 0; i < amountInCart; i++) {
      await this.quantityInput.first().click();
      await this.quantityInput.first().fill("0");
      await this.quantityInput.first().press("Enter");
      await this.page.waitForLoadState("load");
    }
  }
}
```

6. Create Test Using POM

Create `tests/challenge5.spec.ts`:

```
import { test } from "@playwright/test";

// Import page object classes
import { LoginPage } from "./pages/LoginPage";
import { ShopPage } from "./pages/ShopPage";
import { CartPage } from "./pages/CartPage";
import { ProductPage } from "./pages/ProductPage";

test.describe("Challenge 5. Page Object Model", () => {
  // Declare variables for page objects
  let loginPage: LoginPage;
  let shopPage: ShopPage;
  let cartPage: CartPage;
  let productPage: ProductPage;

  // Setup before each test
  test.beforeEach(async ({ page }) => {
    // Initialise page objects
    loginPage = new LoginPage(page);
    shopPage = new ShopPage(page);
    cartPage = new CartPage(page);
    productPage = new ProductPage(page);

    // Launch app and empty cart
    await loginPage.launch();

    await loginPage.authenticate("YOUR_USERNAME_HERE", "YOUR_PASSWORD_HERE");
    await cartPage.emptyCart();
  });

  // Test case: add a product to the cart and verify its contents
  test("Add to Cart and Verify Cart Contents", async () => {
    await shopPage.goToProduct("Useful ChatGPT Prompts");

    await productPage.setProductQuantity(3);
    await productPage.addToCart();

    await cartPage.verifyProductInCart("Useful ChatGPT Prompts", 3);
  });
});
```

7. Run the Test

```
npx playwright test tests/challenge5.spec.ts --headed
```

Challenge 6: End-to-End Flow

Objective

Create a complete end-to-end test that includes login, shopping, and checkout with external test data.

Challenge 6 Checklist

- Installed csv-parse package
- Created User and Product data classes
- Created users.csv and products.csv
- Created dataLoader.ts
- Created CheckoutPage class
- Created OrderPage class
- Created E2EHooks.ts
- Created challenge6.spec.ts
- Updated CartPage with checkout method
- Test loads data from CSV
- Test runs for each user
- Test completes full E2E flow
- Test verifies order success

Step-by-Step Guide

1. Install CSV Parser

```
npm install -D csv-parse
npm install -D @types/csv-parse
```

2. Create Data Classes

Create `tests/dataClasses/Product.ts`:

```
export type Product = {
  name: string;
  price: number;
  quantity: number;
  valid: boolean;
};
```

Create `tests/dataClasses/User.ts`:

```
export type Address = {
  country: string;
  streetAddress: string;
  zipCode: string;
  town: string;
};

export type User = {
  username: string;
  password: string;
  firstName: string;
  lastName: string;
  address: Address;
  emailAddress: string;
  valid: boolean;
};
```

3. Create Test Data Files

Create `tests/testData/users.csv`:

```
username,password,firstName,lastName,country,streetAddress,zipCode,town,emailAddress,valid
Playwright,playwright,John,Doe,Belgium,Zegersdreef 1,2930,Brasschaat,john.doe@example.com,true
```

Create `tests/testData/products.csv`:

```
name,price,quantity,valid
Useful ChatGPT Prompts,0,3,true
```

4. Create Data Loader

Create `tests/support/dataLoader.ts`:

```
import * as fs from 'fs';
import { parse } from 'csv-parse/sync';
import { Product } from '../dataClasses/Product';
import { User } from '../dataClasses/User';

export function loadProductData(): Product[] {
  return parse(fs.readFileSync('./src/testData/products.csv', 'utf-8'), {
    columns: true,
    skip_empty_lines: true
  });
}

export function loadUserData(): User[] {
  return parse(fs.readFileSync('./src/testData/users.csv', 'utf-8'), {
    columns: true,
    skip_empty_lines: true
  }).map((user: any) => {
    return {
      username: user.username,
      password: user.password,
      firstName: user.firstName,
      lastName: user.lastName,
      address: {
        country: user.country,
        streetAddress: user.streetAddress,
        zipCode: user.zipCode,
        town: user.town
      },
      emailAddress: user.emailAddress,
      valid: user.valid === 'true'
    };
  });
}
```

5. Update CartPage to Add Checkout Method

Update `tests/pages/CartPage.ts` to include the checkout method:

```
import { Locator, Page, expect } from "@playwright/test";
import { BasePage } from "./BasePage";

export class CartPage extends BasePage {
  private quantityInput: Locator;
  private checkoutButton: Locator;

  constructor(page: Page) {
    super(page);
    this.quantityInput = page.getByRole("spinbutton", { name: "Product quantity" });
    this.checkoutButton = page.getByRole("link", { name: "Proceed to checkout" });
  }

  async verifyProductInCart(productName: string, expectedQuantity: number) {
    await this.navigateToCart();
    await this.page.waitForLoadState("domcontentloaded");
    await expect(this.page.locator("body")).toContainText(productName);
    await expect(this.page.getByRole("cell", { name: `${productName} quantity` }).getByLabel(
)).toHaveValue(expectedQuantity.toString());
  }

  async emptyCart() {
    await this.navigateToCart();
    await this.page.waitForLoadState("domcontentloaded");
    const amountInCart = await this.quantityInput.count();
    for (let i = 0; i < amountInCart; i++) {
      await this.quantityInput.first().click();
      await this.quantityInput.first().fill("0");
      await this.quantityInput.first().press("Enter");
      await this.page.waitForLoadState("load");
    }
  }

  async checkout() {
    await this.checkoutButton.click();
  }
}
```

6. Create Checkout Page Class

Create `tests/pages/CheckoutPage.ts`:

```
import { Page, Locator } from "@playwright/test";
import { BasePage } from "./BasePage";
import { User } from "../dataClasses/User";

export class CheckoutPage extends BasePage {
  private firstNameInput: Locator;
  private lastNameInput: Locator;
  private countryDropDown: Locator;
  private countryInput: Locator;
  private streetAddressInput: Locator;
  private zipCodeInput: Locator;
  private townInput: Locator;
  private emailAddressInput: Locator;
  private placeOrderButton: Locator;

  constructor(private page: Page) {
    super(page);
    this.firstNameInput = page.getByRole("textbox", { name: "First name" });
    this.lastNameInput = page.getByRole("textbox", { name: "Last name" });
    this.countryDropDown = page.locator("#select2-billing_country-container");
    this.countryInput = page.getByRole("combobox").filter({ hasText: /^$/ });
    this.streetAddressInput = page.getByRole("textbox", { name: "Street address" });
    this.zipCodeInput = page.getByRole("textbox", { name: "Postcode / ZIP" });
    this.townInput = page.getByRole("textbox", { name: "Town / City" });
    this.emailAddressInput = page.getByRole("textbox", { name: "Email address" });
    this.placeOrderButton = page.getByRole("button", { name: "Place order" });
  }
```

```
  // Method to fill billing details form with user data
  async fillDetails(user: User) {
    await this.firstNameInput.click();
    await this.firstNameInput.fill(user.firstName);

    await this.lastNameInput.click();
    await this.lastNameInput.fill(user.lastName);
    await this.countryDropDown.click();
    await this.countryInput.click();
    await this.countryInput.fill(user.address.country);
    await this.page.getByRole("option", { name: user.address.country }).click();

    await this.streetAddressInput.click();
    await this.streetAddressInput.fill(user.address.streetAddress);

    await this.zipCodeInput.click();
    await this.zipCodeInput.fill(user.address.zipCode);

    await this.townInput.click();
    await this.townInput.fill(user.address.town);

    await this.emailAddressInput.click();
    await this.emailAddressInput.fill(user.emailAddress);
  }

  // Method to click the place order button and submit the order
  async placeOrder() {
    await this.placeOrderButton.click();
  }
}
```

7. Create Order Page Class

Create `tests/pages/OrderPage.ts`:

```
import { Page, Locator, expect } from "@playwright/test";
import { BasePage } from "./BasePage";
import { User } from "../dataClasses/User";
import { Product } from "../dataClasses/Product";

export class OrderPage extends BasePage {
  private pageContent: Locator

  constructor(page: Page) {
    super(page);
    this.pageContent = page.locator("body");
  }

  async verifyOrderSuccess(user: User, products: Product[]) {
    await this.page.getByText("Order received").waitFor();

    for (const product of products) {
      await expect(this.pageContent).toContainText(product.name);
    }

    await expect(this.pageContent).toContainText(
      products.reduce((accumulator, currProd) => accumulator + (currProd.price * currProd.quantity), 0).toString()
    );
    await expect(this.pageContent).toContainText(user.firstName);
    await expect(this.pageContent).toContainText(user.lastName);
    await expect(this.pageContent).toContainText(user.emailAddress);
    await expect(this.pageContent).toContainText(user.address.streetAddress);
    await expect(this.pageContent).toContainText(user.address.town);
    await expect(this.pageContent).toContainText(user.address.zipCode);
    await expect(this.pageContent).toContainText(user.address.country);
  }
}
```

8. Create E2E Hooks File

Create `tests/support/E2EHooks.ts`:

```
import { User } from "../dataClass/User";
import { Product } from "../dataClass/Product";
import { loadUserData, loadProductData } from "./dataLoader";
import { test } from "@playwright/test";
import { LoginPage } from "../pages/LoginPage";
import { CheckoutPage } from "../pages/CheckoutPage";
import { OrderPage } from "../pages/OrderPage";
import { ShopPage } from "../pages/ShopPage";
import { ProductPage } from "../pages/ProductPage";
import { CartPage } from "../pages/CartPage";

export let loginPage: LoginPage;
export let checkoutPage: CheckoutPage;
export let orderPage: OrderPage;
export let shopPage: ShopPage;
export let productPage: ProductPage;
export let cartPage: CartPage;

export let userData: User[] = [];
export let ProductData: Product[] = [];

export function setupHooks() {
  userData = loadUserData();
  ProductData = loadProductData();

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    checkoutPage = new CheckoutPage(page);
    orderPage = new OrderPage(page);
    shopPage = new ShopPage(page);
    productPage = new ProductPage(page);
    cartPage = new CartPage(page);

    for (const user of userData) {
      if (user.valid) {
        await loginPage.launch();
        await loginPage.authenticate(user.username, user.password);
        await cartPage.emptyCart();
        await loginPage.logout();
      }
    }
  });
}
```

9. Create E2E Test File

Create `tests/challenge6.spec.ts`:

```
import { test } from "@playwright/test";
import { userData, ProductData, loginPage, shopPage, productPage, cartPage, checkoutPage, orderPage, setupHooks } from
"../support/E2EHooks";

setupHooks();

userData.forEach(user => {
  test(`E2E test for user: ${user.username}`, async () => {
    await loginPage.launch();
    await loginPage.acceptCookies();
    await loginPage.authenticate(user.username, user.password);

    for (const product of ProductData) {
      await shopPage.goToProduct(product.name);

      await productPage.setProductQuantity(product.quantity);
      await productPage.addToCart();
      await cartPage.verifyProductInCart(product.name, product.quantity);
    }

    await cartPage.checkout();

    await checkoutPage.fillDetails(user);
    await checkoutPage.placeOrder();

    await orderPage.verifyOrderSuccess(user, ProductData);
    await loginPage.logout();
  });
});
```

10. Run the E2E Test

```
# Run the E2E test
npx playwright test tests/challenge6.spec.ts

# Run in headed mode
npx playwright test tests/challenge6.spec.ts --headed
```

Congratulations on completing the workshop! You're now equipped with the skills to write professional test automation code. 🎉

Keep testing, keep learning, and keep improving! 🚀